

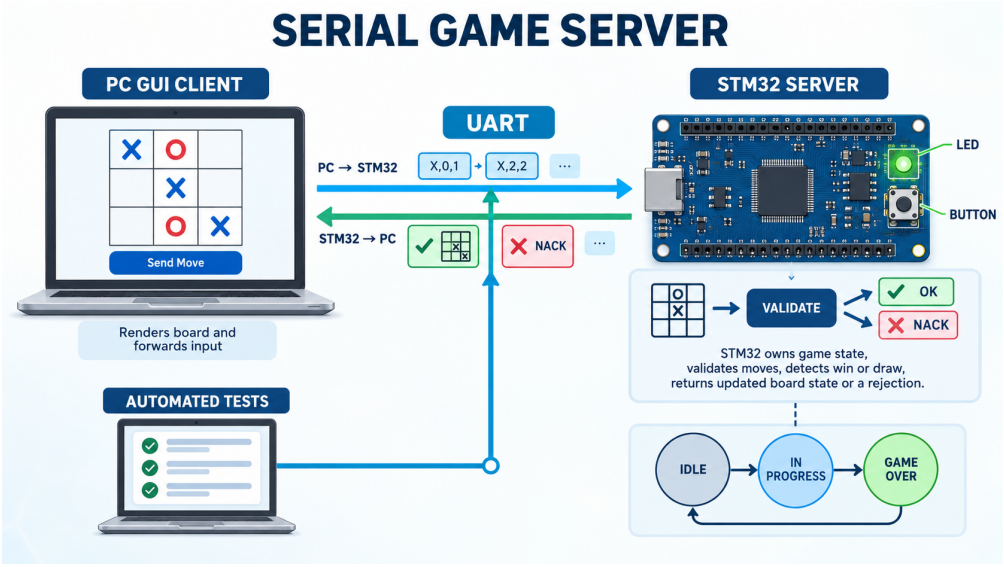
EEE3095S

Embedded Systems II

CS Project 2026

Serial Game Server

An authoritative STM32 game server with a thin PC-side GUI client



System summary: the GUI sends requests; the STM32 validates every action, owns the game state and returns the authoritative result.

DUE	TEAM	VALUE	LATE PENALTY
27 October, 23:59	**Two CS students**	**2 credits**	**10% per day**

Project purpose

Build a small client–server game system over a serial link. The STM32 is the referee and single source of truth; the PC client renders the game and forwards player input. Marks therefore reward sound embedded software, a robust protocol, correct rule enforcement and repeatable validation.

Read the complete brief before beginning implementation. Requirements marked “must” are compulsory.

Project overview

This project fulfils the additional two credits for the EEE3095S version of the course. You will build a small game server on a UCT STM board and a thin PC-side GUI client. The board stores the authoritative game state and enforces every rule. The GUI renders confirmed state and forwards player input; it must not decide whether a move is legal.

The project may be completed by a team of two students. Both team members should be CS students registered for EEE3095S. This project is not part of EEE3096S.

Embedded server	UCT STM board (STM32F051): stores state, validates every move, detects wins and draws, and returns authoritative responses.
PC client	A GUI that connects to the device, displays the board, sends player actions and surfaces results and errors.
Assessment focus	System software, interface design, state-machine correctness, protocol robustness and repeatable tests.

Choosing a game

Choose any turn-based game with a small, well-defined state. Tic-Tac-Toe is the baseline reference. Greater complexity may reach the top band once all compulsory requirements are reliable.

- **Tic-Tac-Toe:** baseline; a 3×3 grid with simple win and draw detection.
- **Connect-4:** column drops, gravity and longer win checks.
- **Mastermind:** a secret held by the server with peg feedback.
- **Battleship:** hidden server-side board state; a strong test of client-untrusted design.

Core principle

The STM32 is the referee. Illegal, out-of-turn and malformed client requests must be rejected without corrupting the server's internal state.

Required system behaviour

Server state machine

The embedded server must implement three clear states:

- **IDLE:** the boot and post-game state; responds to `CONNECT` and `NEW`.
- **IN_PROGRESS:** responds to `MOVE`, `STATE` and `RESIGN`. Every move is validated. A terminal move enters **GAME_OVER**.
- **GAME_OVER:** responds to `NEW`, `STATE` and `DISCONNECT`.

Use the on-board button and LED for a small piece of genuine hardware behaviour. For example, the button may request a local reset/new game, while the LED indicates the current turn or blinks after game over. A server state diagram is compulsory.

PC GUI client

The GUI may be written in Python, C, C++ or a suitable combination. PyQt5, Tkinter, PySimpleGUI and Qt are recommended; document any other library and its installation.

The GUI must support connection, new game, move submission, display of specific move-rejection messages, win/draw notification, disabling moves after a finished game, and clean disconnection. It must redraw the board only from a valid server response.

Embedded STM32 server

Develop the server in STM32CubeIDE using HAL libraries or direct register access. It must:

- communicate over UART serial (or a clearly documented alternative link);
- maintain the board, current turn and game status as the single source of truth;
- reject out-of-range, occupied, out-of-turn and out-of-state moves with specific errors;
- detect all wins and draws and refuse further moves once the game is over;
- frame and parse input safely, including partial, malformed, oversized and garbage data; and
- recover without crashing or blocking forever.

Reference message protocol

For Tic-Tac-Toe, encode the board as nine characters indexed 0–8, using `.` for empty cells and `X`/`O` for players. Messages are ASCII and newline-terminated; a checksum suffix is optional. Adapt and document the protocol for your chosen game.

GUI → STM	**STM → GUI**	**Meaning**
`CONNECT`	`OK`	Establish a session; server is IDLE.
`NEW`	`OK:BOARD:.........`	Start a game and return the empty board.
`MOVE:<index>`	`OK:BOARD:<9 chars>`	Accept a legal move and return confirmed state.
`MOVE:<index>`	`WIN:X / WIN:O / DRAW`	Report a terminal move.
`MOVE:<index>`	`NACK:OCCUPIED / NACK:RANGE / NACK:NOTURN`	Reject an illegal move; state is unchanged.
`STATE`	`BOARD:<9 chars>`	Return the current board.
`RESIGN`	`OK:GAMEOVER`	Forfeit the current game.
`DISCONNECT`	`OK`	End the session.
`malformed / no game`	`NACK:BADCMD / NACK:NOGAME`	Reject an unknown command or invalid game state.

Protocol integrity

Define a terminator and maximum frame length. Clear an incomplete frame after a timeout, issue specific errors, and ensure that every rejected request leaves all game state unchanged. A per-message checksum or CRC is recommended.

Required deliverables

A.1 Project plan

List planned activities and milestones, and show how work is allocated between team members.

A.2 Message protocol description

Define message framing, board encoding, the complete command set and every response/error. Cover new game, move, win/draw and disconnect. A timing or sequence diagram is recommended.

A.3 Circuit diagram

Show the STM UART-to-PC connection, such as the on-board virtual COM port or a suitable converter. Label the on-board button and LED behaviour.

A.4 Design approach

Explain the selected game, GUI library and major design choices, including why responsibilities are placed on the client or server.

A.5 Design diagrams

Provide an embedded server FSM covering IDLE, IN_PROGRESS, GAME_OVER and the move-validation branch, plus a flowchart for the GUI application.

B. Source code

Submit all PC GUI source, the complete embedded STM32 server project/source, and the automated test harness with build and execution instructions.

Automated test harness

Submit a Python test script; `pytest` is recommended. The tests must drive the communication interface and assert, without human interaction, that:

- `CONNECT` returns `OK`, and `NEW` returns an empty board;
- a legal move changes the board exactly as expected;
- an occupied-cell move returns `NACK:OCCUPIED` and leaves the board unchanged;
- an out-of-range move returns `NACK:RANGE`;
- a move after game over returns `NACK:NOGAME` or an equivalent documented error;
- scripted win and draw sequences return the correct terminal result; and
- if an AI opponent is implemented, it never makes an illegal move.

Optional stretch features

Up to five bonus marks are available, with the total capped at 100. Suitable extensions include an AI opponent, two-client play, a hardware-timer move clock, move history/undo, or a hidden-information game that exercises server-side secrecy. Complete and validate the compulsory system first.

Report, submission and demonstration

Submit the project report together with all GUI, STM32 server and automated-test source code. Include concise setup, build and execution instructions so that the work can be reproduced.

The report should contain:

1. **Introduction**: context, selected game and GUI library, task allocation, and report structure.
2. **Requirements and message protocol**: final framing, encoding, command set, errors and justified departures from the reference.
3. **Specification and design**: GUI flowchart, embedded FSM and circuit diagram.
4. **Implementation**: selected, explained code excerpts for important or non-obvious decisions.
5. **Game logic and server authority**: validation, turn control, win/draw detection and proof that rejected requests cannot change state.
6. **Protocol robustness and integrity**: framing, explicit NACKs and recovery from partial or garbage input.
7. **Validation**: automated test results plus screenshots or a short video of the working system.
8. **Conclusion**: success, usefulness, limitations and possible improvements.
9. **Video**: provide a link to a video of your group demonstrating the working of the stm32 serial game server.
10. **Code**: provide a link to the github repo that houses the stm32 serial game server.
11. **References**: relevant libraries, documentation and reused material.

The report format should be:

1. **Format:** IEEE Conference Format; use the template provided at: `https://amathuba.uct.ac.za/d2l/le/lessons/159799/lessons/3570217`
2. **Page Limit**: Maximum 10 pages with all references

The deadline is **27 October at 23:59**. The regular late penalty is **10% per day**.

Before submission
Test the complete archive in a clean folder. Follow your own instructions and confirm that the report, projects, scripts and evidence are present and usable.

Project marking guidelines

Heading	Evidence expected	Marks
Introduction	About half a page. Explain the main design choices, task allocation and report structure.	10
Requirements and message protocol	Describe and diagram the final protocol, including framing, board encoding and error responses. Highlight departures from the brief.	15
Specification and design	GUI and embedded flowchart/FSM diagrams plus the circuit diagram. Diagrams may be at function level.	15
Implementation	Explain key code excerpts, especially parts that were not straightforward to implement.	15
Game logic and server authority	Correct move validation, rejection of illegal/out-of-turn moves, reliable win/draw detection and a client-untrusted design.	15
Protocol robustness and integrity	Framing, specific NACKs for malformed input, and graceful recovery from partial or garbage data.	10
Validation and automated test harness	Passing tests for the required cases, plus screenshots or a short video of the working system.	15
Conclusion	Summarise success, usefulness and possible improvements.	5
References	A few relevant references, including the GUI library used.	–
Total		**100**

Bonus marks
Up to five bonus marks may be awarded for the listed stretch features. The final project mark remains capped at 100.